

WMS: System Architecture

Wholesale/retail warehouse management system · Branch `fix/audit-remediation` · As of 19 Sep 2026 · Live version `06c0e016`

1. Overview

The WMS is one Next.js application built from `WMS_Wholesale_Retail_PRD.md`. It serves five role-based portals and uses one backend and one Postgres database. The app runs on Cloudflare Workers, and the database runs on Neon in Singapore, the Neon region closest to the warehouse in India.

All five portals share one design system, and your role decides which screens you can see. No screen uses decorative or animated elements.

Portal	Path	Role	UI style
Admin	<code>/admin</code>	ADMIN	Clean desktop ERP: users, settings, catalogue, backups
Manager	<code>/manager</code>	MANAGER	Clean desktop ERP: inventory, suppliers, reports, cash day book, order oversight
Finance (the till)	<code>/finance</code>	FINANCE	Dense and keyboard-first, like MARG/Tally. Built for fast billing, with shortcuts such as Alt+N, Alt+C and Alt+L
QC	<code>/qc</code>	QC	Touch-first and responsive. One build serves desktop and tablet
Payment Display	<code>/pay</code>	none (kiosk)	Minimal, full-screen display facing the customer

2. Runtime topology

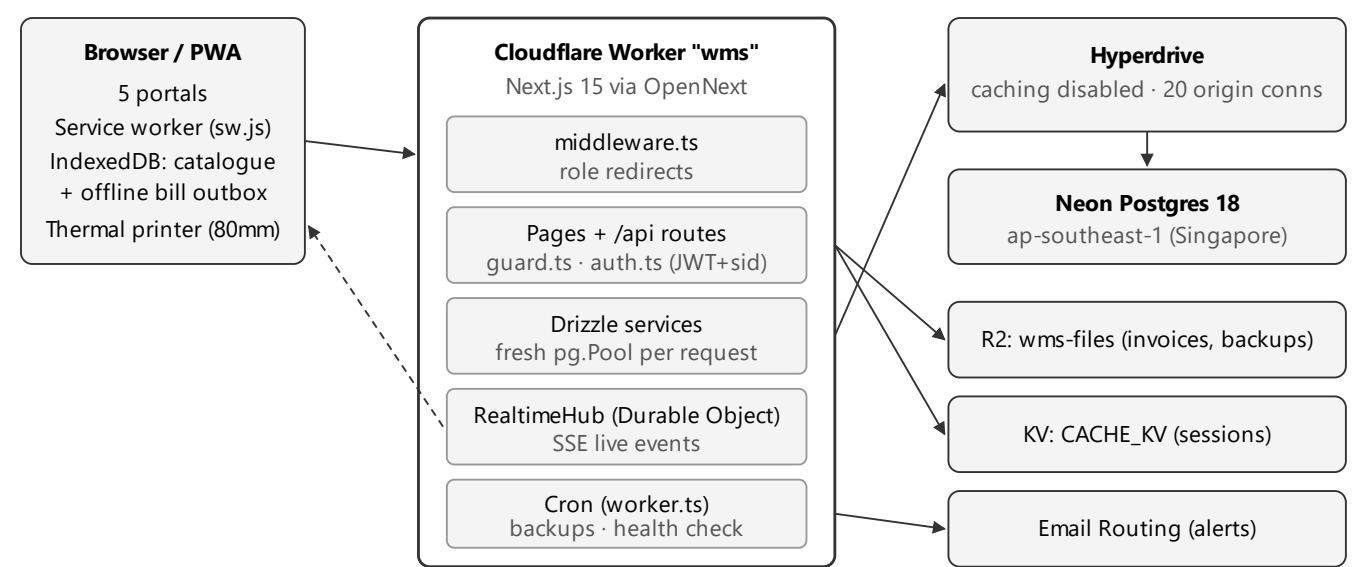


Figure 1. Request path from the browser through the Worker to data and platform services. The dashed arrow is the SSE push back to clients.

Domains

- Primary: `taresh.harshuthecoder.qzz.io` . Its DNS is on Cloudflare, and it routes to the whole Worker.
- Fallback: `wms.shpstack.workers.dev` .

Cloudflare bindings (`wrangler.jsonc`)

Binding	Type	Purpose
<code>HYPERDRIVE</code>	Hyperdrive	Pooled connection to Neon. Result caching is disabled ; with caching on, a read right after a write returns stale data.
<code>FILES</code>	R2 <code>wms-files</code>	Invoice HTML, product images, daily DB backups (30-day pruning)
<code>NEXT_INC_CACHE_R2_BUCKET</code>	R2 <code>wms-opennext-cache</code>	OpenNext incremental cache
<code>CACHE_KV</code>	KV	Single-device login registry, <code>session:<userId> → {sid}</code>
<code>REALTIME_HUB</code>	Durable Object	Fans live events out to portals over SSE. Scopes are keyed by warehouse, role, order and user.
<code>EMAIL</code>	send_email	Down/recovered alerts from the 5-minute health-check cron
<code>ASSETS</code> , <code>WORKER_SELF_REFERENCE</code>	OpenNext	Static assets and self-calls

3. Data access: two query layers

The same Postgres schema is reached through two ORMs. `isWorkersRuntime()` in `src/lib/cf-env.ts` picks one on each request:

Layer	Where it runs	Files
Prisma	Local next dev only	<code>prisma/schema.prisma</code> , <code>src/generated/prisma</code> , <code>src/lib/db.ts</code> , <code>*-service.ts</code>
Drizzle	Cloudflare Workers (production)	<code>src/generated/drizzle</code> (produced by introspecting the DB; not tracked in git), <code>src/lib/drizzle-db.ts</code> , <code>drizzle-*.ts</code>

- Prisma's query engine can't run on Workers at all, because the platform blocks WASM compilation. Routes therefore load the Drizzle path with `await import()`, so Prisma is never loaded in production.
- Prisma owns the schema: migrations live in `prisma/migrations`. After a migration, `npm run db:deploy` runs `drizzle-kit introspect` so the Drizzle schema follows it.
- Where both layers run the same logic, the raw SQL is written once and executed through Prisma `$queryRaw` or the Drizzle `sql` tag. This applies to the cash drawer, the day book and the customer billing context.

Never cache the `pg.Pool` on Workers. Isolates recycle sockets between requests, so a cached pool makes requests hang. `getDrizzleDb()` creates a new pool (`max: 1`) for each request and retries the connection with backoff. Hyperdrive allows at most 20 connections to the origin.

4. Security and sessions

- **Auth:** a JWT signed with `jose` in `src/lib/auth.ts`, carrying a `sid` claim. Passwords are hashed with `bcrypt`.
- **One device per account:** each `getSession()` call checks the `sid` against KV. If KV can't be reached, the check lets the request through, so a KV outage can't lock out the warehouse. When a second device logs in, the first device gets 20 seconds to deny the takeover; if nobody answers, the new device wins. The prompt reaches the old device through `SessionGuard.tsx` over SSE.
- **Authorisation:** `middleware.ts` redirects each role to its own portal, and `guard.ts` checks the role inside every API route. Finance and Manager reads are limited to the viewer's warehouse; ADMIN sees all warehouses.
- **Edge protection:** a WAF rate limit on `/api/auth/login` (5 requests per minute per IP).
- **Outbound calls:** the GSTIN lookup accepts only https URLs, rejects private and loopback hosts, and substitutes the API key on the server so it never reaches a browser.

5. Business domains

Domain	Key modules (src/lib)	Notes
Billing	<code>invoice.ts</code> , <code>bill-revision.ts</code> , <code>bill-lines.ts</code> , <code>numbering.ts</code> , <code>pricing.ts</code>	Bills are paid in cash on creation unless the clerk ticks "Unpaid". The printed estimate is 72mm wide with lines sorted A→Z. Modify Bill can add or remove lines on an issued bill. Bills can be held (Alt+1...9) or saved as drafts.
Inventory	<code>inventory.ts</code> , <code>drizzle-inventory.ts</code> , <code>stock.ts</code> , <code>units.ts</code> , <code>base-unit.ts</code> , <code>purchase.ts</code>	Reservations are batched, and locks are taken in <code>productId</code> order so two tills can't deadlock. Stock is netted per product in base units. Supplier purchase bills and GRNs are supported, and managers can reset stock or edit reserved quantities.
QC	<code>qc-service.ts</code> , <code>qc-lock.ts</code> , <code>complete-without-qc.ts</code>	Only one QC user can work an order, with a 15-minute idle timeout; a manager can force-release the lock. QC can only reduce quantities or mark lines unavailable, and each change needs a reason. There is no approval step.
Payments and cash	<code>payment-service.ts</code> , <code>cash.ts</code> , <code>daybook.ts</code> , <code>mark-unpaid.ts</code> , <code>upi.ts</code>	Payment rows are locked with <code>SELECT ... FOR UPDATE</code> , and collecting twice returns 409. Each user has a chain of drawer sessions. Finance counts the drawer blind (the expected amount appears only after saving). The manager's day book reconciles the drawer against the bank statement.
GST invoicing	<code>gst.ts</code> , <code>gst-invoice.ts</code> , <code>gst-register.ts</code> , <code>gstin-lookup.ts</code>	A GST invoice stands alone, and its <code>AuditLog</code> row is the record. GSTINs are validated with the checksum. Invoices are revised by issuing a new number that supersedes the old one. The register can export to CSV.
Audit and notifications	<code>audit.ts</code> , <code>notifications.ts</code> , <code>realtime.ts</code> , <code>live-events.ts</code>	Every mutation writes an audit row. Portals receive live updates through the RealtimeHub.

Order lifecycle

DRAFT → billed at the Finance till (stock reserved, paid in cash) → QC (may reduce lines, which triggers a bill revision and a refund or extra payment) → complete. An order can be cancelled before QC. Cancelling releases its reservations, and a popup records how the money went back: cash, UPI, credit, or later.

6. Offline-first operation

- **PWA:** installable through `public/manifest.json`. `public/sw.js` serves `/_next/static` cache-first, serves other pages network-first with a cache fallback, and never caches `/api/*` or RSC requests. The `/offline` fallback page is cached at install.
- **Read cache:** `GET /api/sync/catalog` sends products, customers and units to IndexedDB (`offline-catalog.ts`). Search shows cached results immediately, then updates them with live stock.
- **Write outbox:** bills, cash and Modify-Bill changes are saved to IndexedDB *before* any network call (`offline-bills.ts`) and synced in order. Nothing is deleted from the outbox until the server confirms it. If the server doesn't answer within 3 seconds, the till prints a provisional slip numbered `OFF-<STAFF>-<DEVICE>-<MMDD>-<NNN>`.
- **Idempotency:** each request carries a `clientRequestId` generated on the client. The server records it through the `IdempotencyKey` table and `Order.clientRequestId`, so a retry of a committed request returns `duplicate: true` instead of saving it twice.
- **Online only:** sending to QC, QC sessions, UPI and all admin configuration still need a live connection.

7. Scheduled jobs (`worker.ts`)

- Daily database backup to R2, with backups older than 30 days pruned. Only ADMIN can download backups.
- A health check every 5 minutes that emails when the app goes down and when it recovers.

8. Build, deploy and operations

Command	Purpose
<code>npm run dev</code>	Local development (Prisma path, Cloudflare binding mocks, local Postgres)
<code>npm test</code>	Unit suite, about 130 checks (<code>src/lib/__tests__/run.ts</code>)
<code>npm run test:db</code>	Tests stock-reservation concurrency against a real Postgres
<code>npm run db:deploy</code>	Applies Prisma migrations, then re-introspects the Drizzle schema
<code>npm run cf:deploy</code>	OpenNext build, then deploy to Cloudflare
<code>npm run smoke</code>	Post-deploy check (4 checks: routing, auth, DB reachability)
<code>scripts/offline-e2e/</code>	Browser harness: 15 offline scenarios

Deploy procedure

1. Apply any new migration to Neon first. Use `prisma migrate deploy` against the **direct** endpoint, not the pooler.
2. Kill every node, workerd and esbuild process running from the project directory.
3. Delete only `.open-next`. Keep `.next`: on this machine, a build that starts from an empty `.next` fails.
4. Run `npm run cf:deploy` from the main project directory. **Never deploy from a git worktree or the mirror directory:** both have produced Workers that return 500 on every request.

5. Run `npm run smoke` straight away. If it fails, roll back with `npx wrangler versions deploy <id>@100% --yes`.

Logging: Workers observability is enabled with full sampling. `npx wrangler tail --format pretty` gives live logs.

9. Known issues and open items

- **CPU-limit error:** `POST /api/finance/orders` sometimes fails with "Exceeded CPU Limit". The leading suspect is Drizzle rebuilding its relational config on every request.
- **GST blocked:** `BUSINESS_GSTIN` is not set in production, so no GST invoice can be issued. GSTIN autofill also needs a provider API key.
- **GST bills record no payment**, so their cash reaches neither the drawer nor the bank expectation.
- **Invoices are overwritten on revision:** the stored invoice HTML key has no version number, so a revision replaces the file. The `BillVersion` data is kept.
- **Shipped uncommitted:** the working tree is ahead of the last commit (`7e63cb1`). The live version was deployed from uncommitted work.
- **No authenticated test in production:** nobody has logged in and run the EOD, GST or paid-on-creation flows against production.
- **Outdated harness expectations:** 5 of the 15 offline e2e scenarios fail because their expectations predate the paid-on-creation and EOD changes.